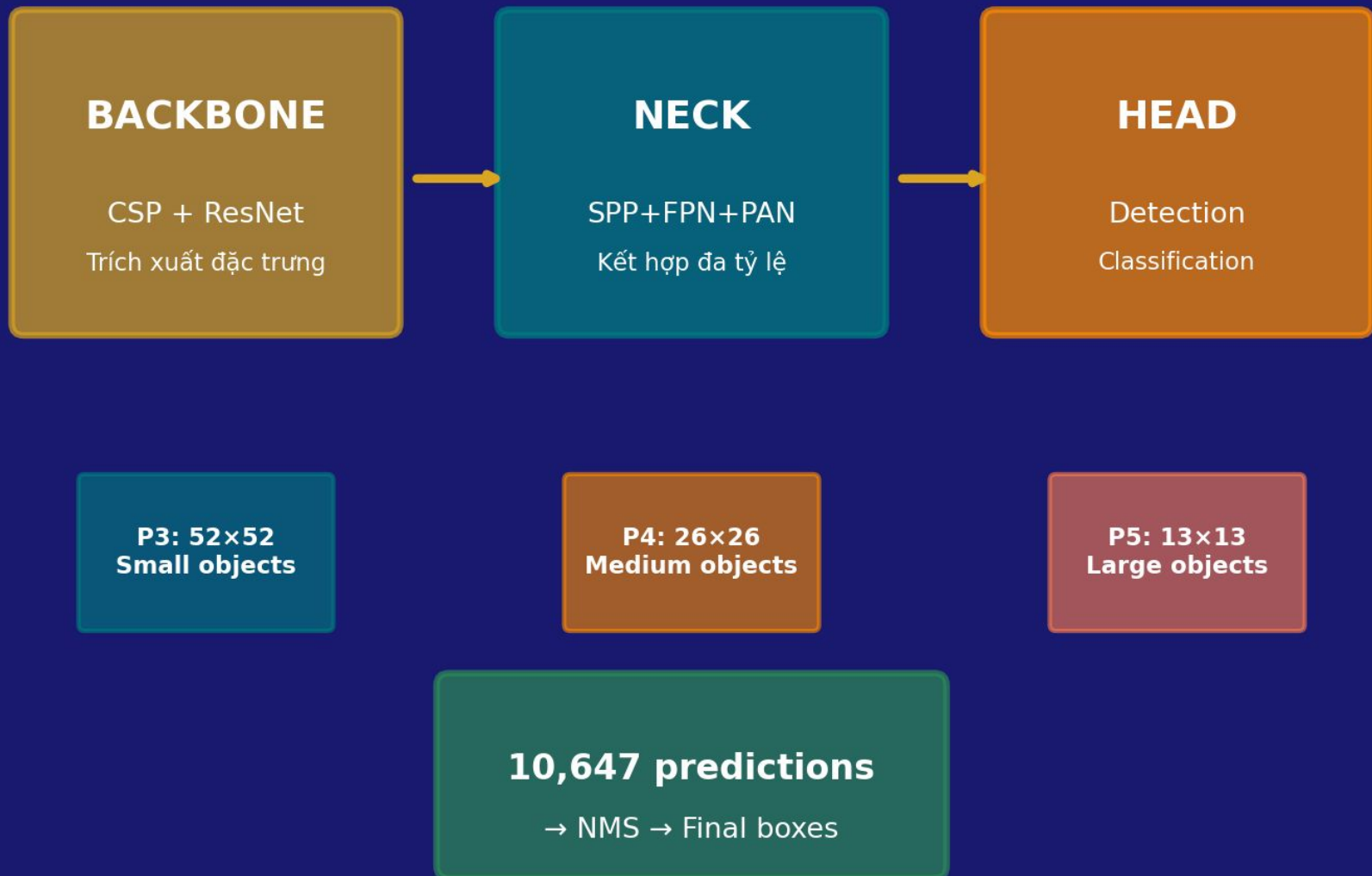


KIẾN TRÚC MẠNG

YoNet

Phát Hiện Đồng Hồ Nước Trên FPGA

TỔNG QUAN KIẾN TRÚC



1. CONVOLUTION 2D - PHÉP TÍCH CHẬP

$$Y[b, c_{out}, h, w] = \sum_{c_{in}} \sum_i \sum_j X[b, c_{in}, h \cdot s + i, w \cdot s + j] \cdot W[c_{out}, c_{in}, i, j] + \text{bias}$$



Trích xuất đặc trưng cục bộ • Parameter sharing • Receptive field

2. BATCH NORMALIZATION

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$y = \gamma \cdot \hat{x} + \beta$$

Ổn định phân phối activations

Training nhanh hơn, learning rate cao được

DPU có hardware accelerator

Fold vào Conv khi inference → 1 operation

3. HÀM KÍCH HOẠT

$$f(x) = \max(0.1015625 \cdot x, x)$$

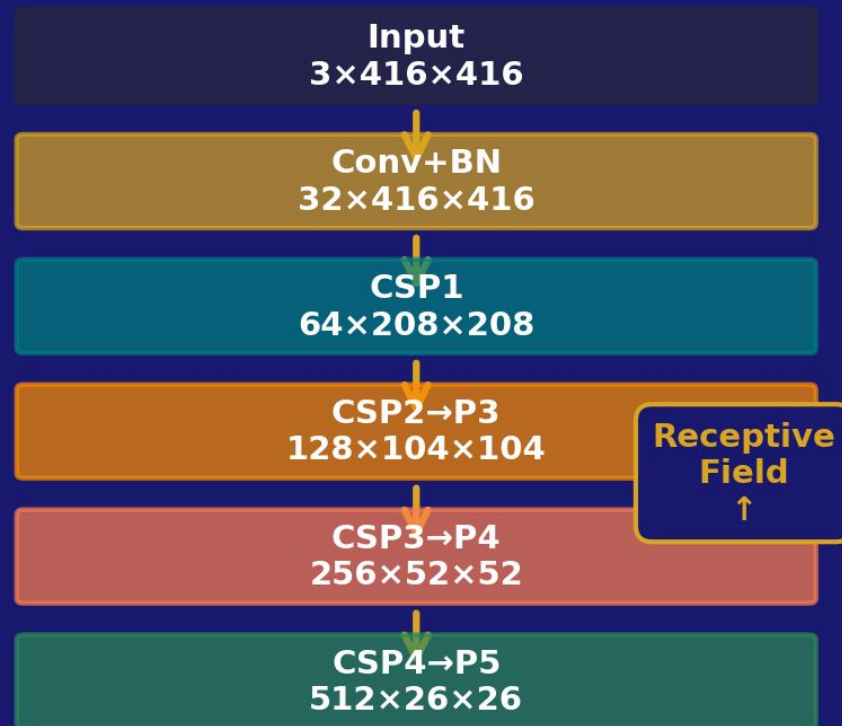
$$\sigma(x) \approx 0.5 + 0.25x - 0.021x^3$$

LeakyReLU: Tránh dying neurons, slope tối ưu cho DPU

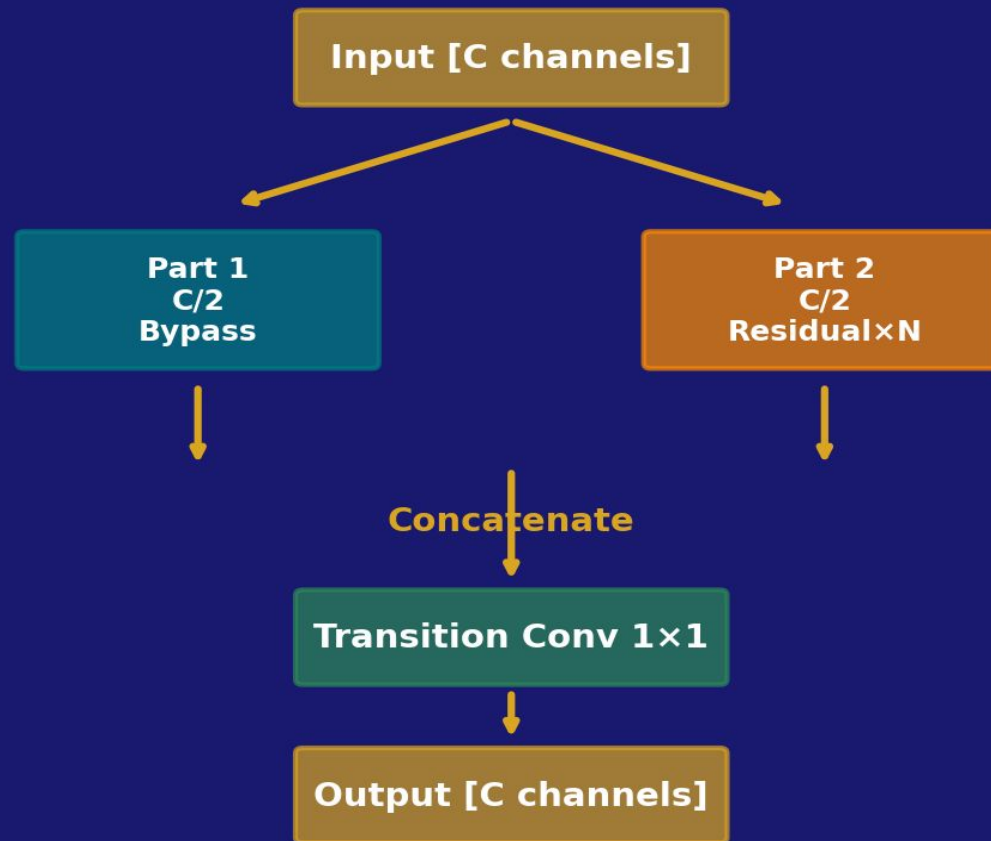
Sigmoid: Polynomial thay e^x , speedup 3-4x

Tại sao cần Activation?
Không có → chỉ là hàm tuyến tính
Có activation → học patterns phức tạp

BACKBONE - XỬ LÝ SỐNG MẠNG



CSP BLOCK - CROSS STAGE PARTIAL



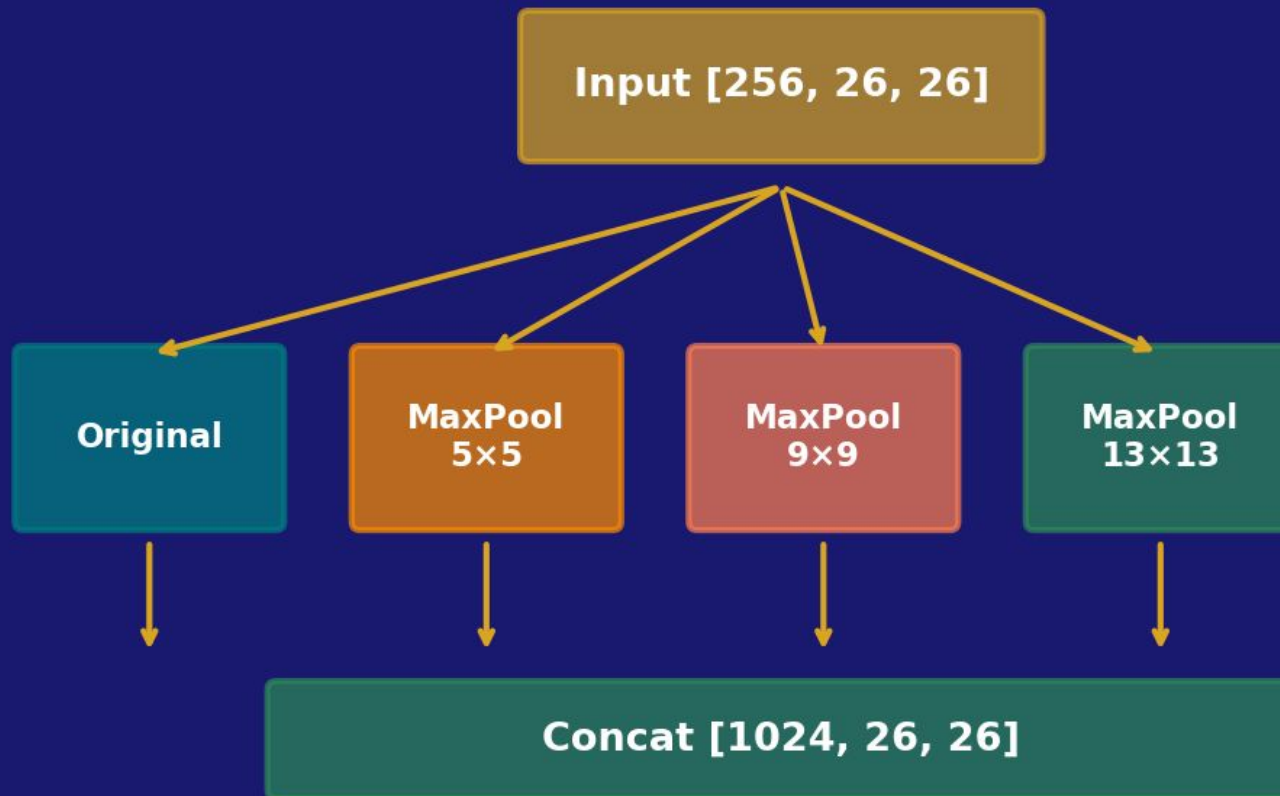
TẠI SAO CẦN NECK?

Vấn đề: Features độc lập, thiếu tương tác!

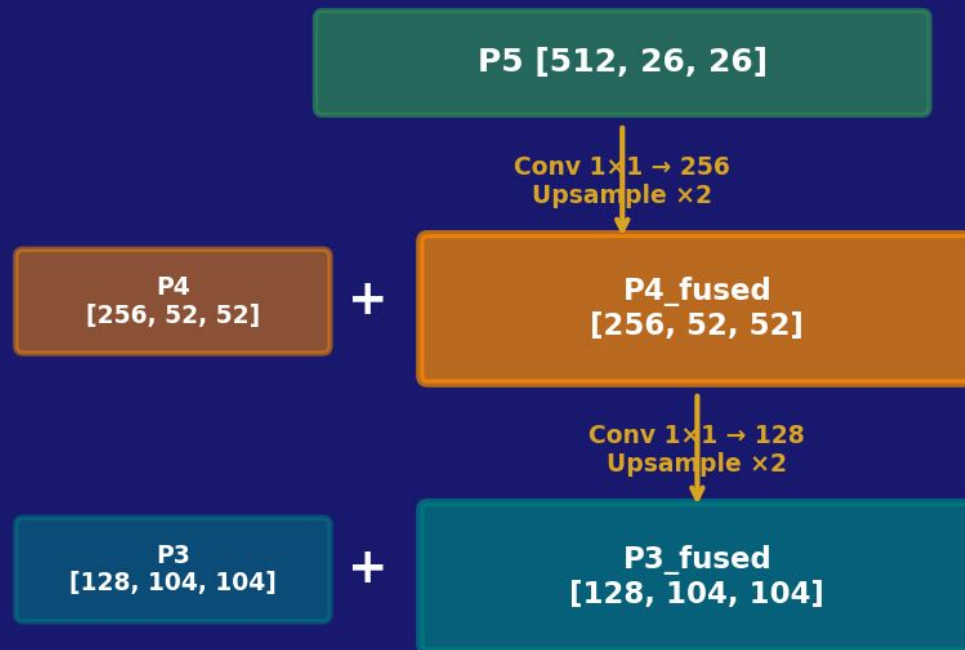
Level	Resolution	Semantics	Localization
P3	104×104	Yếu	Mạnh
P4	52×52	Trung bình	Trung bình
P5	26×26	Mạnh	Yếu

Giải pháp: FPN (semantics) + PAN (localization)

SPP - SPATIAL PYRAMID POOLING



FPN - TOP-DOWN SEMANTIC FLOW



FPN: Top-Down Semantic Flow

PAN - BOTTOM-UP LOCALIZATION

P3_fused [128, 104, 104]

↓ Conv stride=2

P4 + P3↓ → P4_refined [256, 52, 52]

↓ Conv stride=2

P5 + P4↓ → P5_refined [512, 26, 26]

Luồng 2 chiều:
FPN (top-down) + PAN (bottom-up)
= Semantics + Localization!

CHANNEL ATTENTION - SE BLOCK

$$a_c = \sigma(W_2 \cdot \text{ReLU}(W_1 \cdot z_c))$$

Global Average Pooling: $C \times H \times W \rightarrow C \times 1 \times 1$

Excitation: $C \rightarrow C/16 \rightarrow C$ (2 FC layers)

Sigmoid: Attention weights $[0,1]$

Scale: $Y = X \cdot \text{attention}$

Ví dụ:

Channel 245 (meter): 0.95 ✓

Channel 246 (digit): 0.98 ✓✓

Channel 500 (noise): 0.05 ✗

DETECTION HEAD - DỰ ĐOÁN

Separate Paths: Detection $\neq$ Classification

Detection Path
2 Conv layers
Box regression

Classification Path
3 Conv layers
Class logits

$$b_x = (\sigma(t_x) + c_x) \cdot \text{stride}, \quad b_w = a_w \cdot e^{t_w}$$

Output: (tx,ty,tw,th,obj,c1,...,c11)
3 anchors $\times$ 3 scales = 9 predictions/grid

NMS - NON-MAXIMUM SUPPRESSION

$$\text{IoU} = \frac{\text{Area(Intersection)}}{\text{Area(Union)}}$$

1. Lọc theo threshold: 10,647 $\rightarrow$ $\sim$ 500 boxes
2. Sort theo confidence (giảm dần)
3. Loại boxes trùng: $\text{IoU} > 0.45$
4. Output: $\sim$ 10-50 boxes cuối cùng

HARDWARE OPTIMIZATION - DPU

Fixed Sizes

Grid 13,26,52

Không dynamic

LeakyReLU

Slope DPU-friendly

0.1015625

Power-of-2

32,64,128,256,512

SIMD vectorize

BatchNorm

DPU accelerated

Fold vào Conv

Sigmoid Approx

Polynomial

Speedup 3-4x

INT8 Quant

4x nhỏ, 4x nhanh

-0.3% mAP

INT8 QUANTIZATION

$$W_{\text{int8}} = \text{round}\left(\frac{W_{\text{fp32}}}{\text{scale}}\right), \quad W_{\text{fp32}} \approx W_{\text{int8}} \cdot \text{scale}$$

FP32 → INT8
97 MB → 24 MB
4× compression!

Performance
4× faster
-0.3% mAP only

Calibration: 100-500 ảnh → tính scale factors

Vitis AI Quantizer → Compiler → .xmodel cho DPU

PERFORMANCE - HIỆU NĂNG

Component	Time	%
Backbone	18.3ms	52%
SPP	2.1ms	6%
FPN	4.2ms	12%
PAN	6.8ms	19%
Attention	0.8ms	2%
Heads	2.5ms	7%
Post-process	0.6ms	2%
TOTAL	35.3ms	28 FPS

SO SÁNH: GPU vs DPU

NVIDIA T4 GPU

FPS: 161 (INT8)

Latency: 6.2ms

Power: 70W ✗

Cost: \$2,000 ✗

Cần host PC ✗

Xilinx DPU

FPS: 28 (INT8)

Latency: 35ms

Power: 5W ✓

Cost: <\$100 ✓

Standalone ✓

Use case DPU:

Đồng hồ nước IoT • Camera giám sát
Robot công nghiệp • Xe tự hành

KẾT QUẢ THỰC NGHIỆM

Metric	Value
mAP@0.5	61.8%
mAP@0.75	42.7%
Small objects	48.1%
Medium objects	64.9%
Large objects	76.3%
FPS (DPU)	28
Latency	35.3ms
Power	5W

NGUYÊN TẮC THIẾT KẾ YoNet

Multi-scale

3 levels: P3,P4,P5

Bidirectional

FPN + PAN

Efficiency

CSP: 4.2× faster

Attention

Channel recalibration

Hardware-aware

INT8, DPU-friendly

Balanced

61.8% @ 28 FPS

KẾT LUẬN

YoNet = CNN hiện đại cho Edge AI

BACKBONE (CSP + ResNet)

→ Trích xuất features hiệu quả

NECK (SPP + FPN + PAN)

→ Kết hợp đa tỷ lệ, 2 chiều

HEAD (Separate paths)

→ Detection & Classification tối ưu

HARDWARE OPTIMIZATION

→ INT8, DPU-friendly, 5W @ 28 FPS

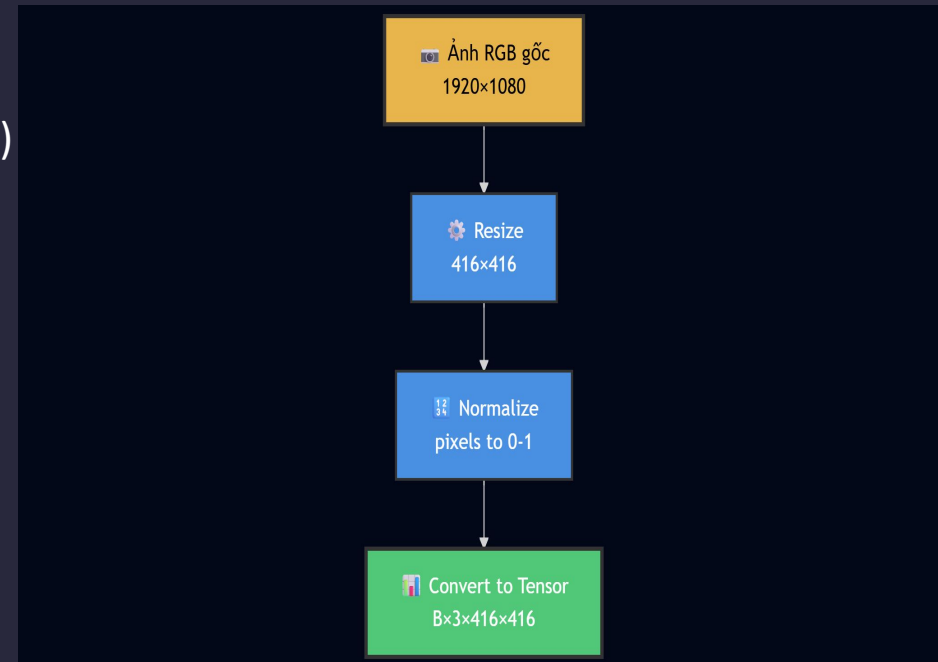
LUỒNG XỬ LÝ - BƯỚC 1: INPUT

Ảnh đầu vào:

- Ảnh RGB từ camera hoặc file
- Kích thước gốc: bất kỳ (ví dụ: 1920×1080)

Preprocessing:

1. Resize → 416×416 pixels
2. Normalize pixel values → [0, 1]
3. Convert to Tensor → [B, 3, 416, 416]
 - B = batch size
 - 3 = RGB channels
 - 416 = height, width



Input Tensor: $X \in \mathbb{R}^{B \times 3 \times 416 \times 416}$

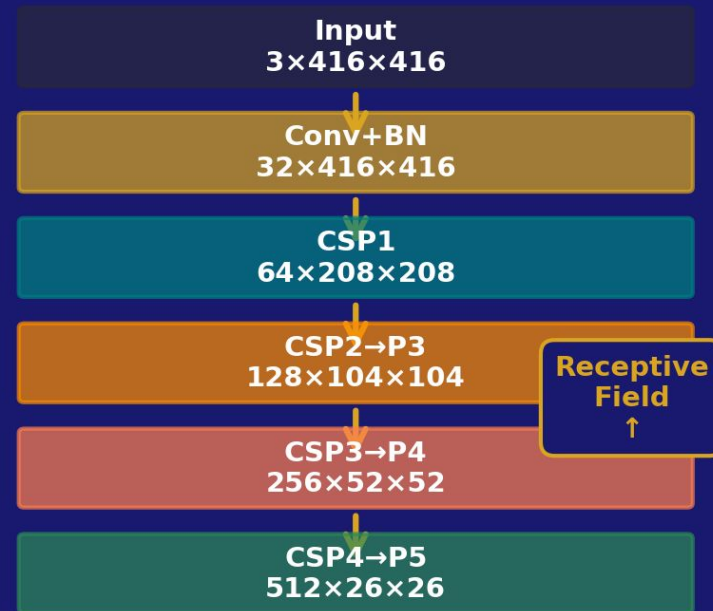
BƯỚC 2: BACKBONE - TRÍCH XUẤT ĐẶC TRƯNG

Backbone output - 3 feature maps:

- P3: [B, 128, 104, 104] - Small objects
- P4: [B, 256, 52, 52] - Medium objects
- P5: [B, 512, 26, 26] - Large objects

Mỗi feature map chứa:

- Semantic information
- Spatial structure



BƯỚC 3: NECK - FUSION ĐA TỶ LỆ

SPP Block (Spatial Pyramid Pooling):

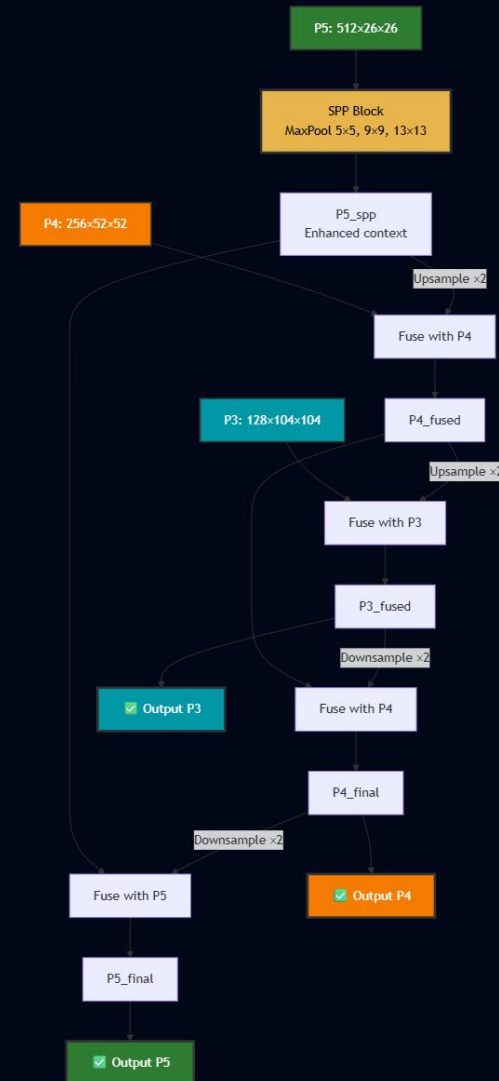
- $P5 \rightarrow SPP \rightarrow$ Context features
- MaxPool: 5×5 , 9×9 , 13×13
- Tổng: $80 + 144 + 208 = 432$ pixels context

FPN (Top-Down Pathway):

- $P5_spp \rightarrow$ upsample $\rightarrow$ fuse with $P4$
- $P4_fused \rightarrow$ upsample $\rightarrow$ fuse with $P3$
- Semantic info flows: deep $\rightarrow$ shallow

PAN (Bottom-Up Pathway):

- $P3_fused \rightarrow$ downsample $\rightarrow$ fuse with $P4$
- $P4_fused \rightarrow$ downsample $\rightarrow$ fuse with $P5$
- Localization info flows: shallow $\rightarrow$ deep



BƯỚC 4: DETECTION HEAD - PREDICTIONS

Mỗi scale có 3 anchor boxes:

- P3: [B, 3, 104, 104, 5+C]
- P4: [B, 3, 52, 52, 5+C]
- P5: [B, 3, 26, 26, 5+C]

Output: [B, 3, S, S, 5 + C] với $S \in \{13, 26, 52\}$

Mỗi prediction box gồm:

1. objectness (1): Có object hay không?
2. x, y (2): Tọa độ tâm box (relative to cell)
3. w, h (2): Kích thước box (relative to anchor)
4. class probs (C): Xác suất từng class

Total predictions = $3 \times (104^2 + 52^2 + 26^2) = 40,572$ boxes!

BƯỚC 5: POST-PROCESSING

1. Confidence Thresholding:

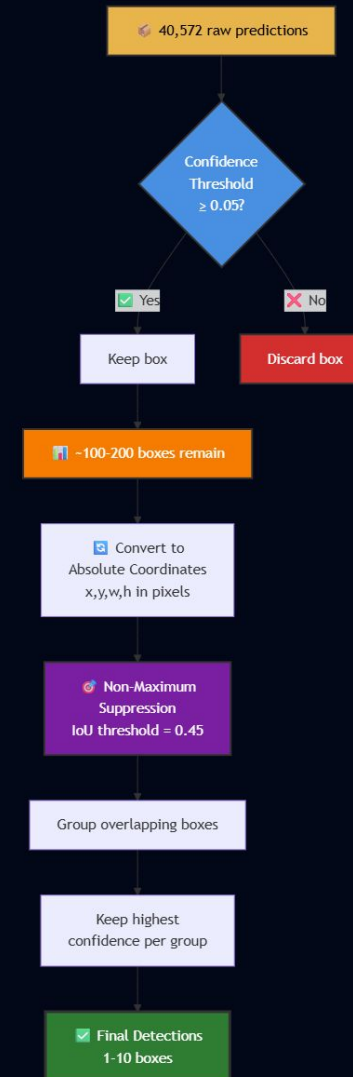
- Loại bỏ predictions với objectness < 0.05
- 40,572 boxes $\rightarrow$ ~ 100 -200 boxes

2. Convert to Absolute Coordinates:

- (x, y) cell-relative $\rightarrow$ pixel coordinates
- (w, h) anchor-relative $\rightarrow$ pixel size

3. Non-Maximum Suppression (NMS):

- Group overlapping boxes
- Keep highest confidence box
- IoU threshold = 0.45
- ~ 100 boxes $\rightarrow$ 1-10 final detections



BƯỚC 6: LOSS CALCULATION (TRAINING)

So sánh predictions với ground truth:

1. Assign ground truth boxes to anchors:

- Dựa trên IoU giữa GT box và anchor shapes
- Best anchor → responsible for prediction

2. Calculate 4 loss components:

- Box loss: MSE cho (x, y, w, h)
- Object loss: BCE cho cells có object
- No-object loss: BCE cho cells không có object
- Class loss: CrossEntropy cho classification

3. Multi-scale total loss:

$$L_{\text{total}} = L[P3] + L[P4] + L[P5]$$



LUỒNG HOÀN CHỈNH - END-TO-END

Tóm tắt toàn bộ pipeline:

Input: [B, 3, 416, 416]

↓ Backbone (CSP + ResNet)

P3, P4, P5 features

↓ Neck (SPP + FPN + PAN)

Enhanced multi-scale features

↓ Detection Head (3 branches)

Raw predictions: 40,572 boxes

↓ Post-processing (Threshold + NMS)

Final detections: 1-10 boxes

↓ Loss Calculation (Training only)

Gradients → Backprop → Update weights



Inference: ~35ms | Training: ~50ms/image

HÀM MẤT MÁT - TỔNG QUAN

Loss Function là gì?

- Đánh giá độ sai lệch giữa dự đoán và ground truth
- Hướng dẫn model học đúng hướng

YOLO Loss bao gồm 4 thành phần:

1. Box Coordinate Loss (λ_{box})
2. Object Confidence Loss (λ_{obj})
3. No-Object Confidence Loss (λ_{noobj})
4. Classification Loss (λ_{class})

$$\text{Total Loss} = \lambda_{box} \cdot L_{box} + \lambda_{obj} \cdot L_{obj} + \lambda_{noobj} \cdot L_{noobj} + \lambda_{class} \cdot L_{class}$$

1. BOX COORDINATE LOSS

$$\mathcal{L}_{\text{box}} = \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right]$$

Mean Squared Error cho tọa độ bounding box

- (x, y) : Tọa độ tâm - sigmoid activation
- (w, h) : Kích thước - logarithm space
- $\sqrt{w}, \sqrt{h}$: Cân bằng giữa box lớn và nhỏ

Tại sao dùng log space cho w, h ?

- Ổn định gradient
- Box nhỏ và lớn có loss tương đương

2. OBJECT & NO-OBJECT LOSS

$$\mathcal{L}_{\text{obj}} = \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \text{BCE}(C_i, \hat{C}_i \cdot \text{IoU}_{\text{pred}}^{\text{truth}})$$

$$\mathcal{L}_{\text{noobj}} = \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} \text{BCE}(C_i, \hat{C}_i)$$

BCE = Binary Cross Entropy Loss

- Object loss: Weighted by IoU with ground truth
- No-object loss: Cực kỳ quan trọng ($\lambda_{\text{noobj}} = 10.0$)

Tại sao λ_{noobj} lớn?

→ 99% cells không có object → cần penalize mạnh

3. CLASSIFICATION LOSS

$$\mathcal{L}_{\text{class}} = \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} -\log(p_i(c)) \cdot \mathbb{1}_c^{\text{true}}$$

Cross Entropy Loss cho phân loại đa lớp

- Chỉ áp dụng cho cells có object
- Softmax trên C classes
- Maximize log-likelihood cho class đúng

Ví dụ với Water Meter Detection:

- Class 0: Analog meter
- Class 1: Digital meter
- Class 2: Smart meter

WEIGHTED LOSS - CÂN BẰNG

$\lambda_{\text{box}} = 10.0$

Box coordinates

$\lambda_{\text{obj}} = 1.0$

Object confidence

$\lambda_{\text{noobj}} = 10.0$

No-object confidence

$\lambda_{\text{class}} = 1.0$

Classification

Tại sao cần weights?

- Box loss: Quan trọng cho localization
- No-obj loss: Cân bằng imbalance (99% cells)
- Weights được tune qua Optuna HPO

HỆ THỐNG LOSS HOÀN CHỈNH

$$\mathcal{L}_{\text{total}} = \lambda_{\text{box}}\mathcal{L}_{\text{box}} + \lambda_{\text{obj}}\mathcal{L}_{\text{obj}} + \lambda_{\text{noobj}}\mathcal{L}_{\text{noobj}} + \lambda_{\text{class}}\mathcal{L}_{\text{class}}$$

Training với PyTorch Ignite:

- Mixed Precision (AMP) $\rightarrow$ 2 $\times$ faster
- Gradient Scaler $\rightarrow$ Tránh underflow
- Multi-scale loss: $L = L[P3] + L[P4] + L[P5]$

Optimization:

- Adam optimizer (lr=1e-4, weight_decay=1e-4)
- OneCycleLR scheduler
- Warmup: 5 epochs
- Early stopping based on composite score

CẢM ƠN!

Q&A - Hỏi Đáp